

Dynamic Image Gallery with Effects

Submitted for Web Programming Minor Project

of 4th Semester

Master of Computer Applications with specialization in
Artificial Intelligence and Internet of Things – CSE



Submitted By:

Jaskaran Singh (2447018)
Prashant Kumar Mishra (2447021)
Bhanu Katoch (2447022)
Ajay Kumar (2447023)
Nikita Mewada (2447027)
Saptarishi Das (2447028)

Submitted to: Dr. Adla Sanober

Course Title: Web Programming (MC470404)

Department of Computer Science & Engineering

NATIONAL INSTITUTE OF TECHNOLOGY PATNA

University Campus, Bihar- 800005

July 2025- December 2025

Contents

- 1 Project Title
- 2 Abstract
- 3 Introduction
- 4 Objectives
- 5 Theory
- 6 Algorithm
- 7 Implementation
- 8 Result
- 9 Conclusion
- 10 Individual Contribution
- 11 References

Project Title

This project is titled "**Dynamic Image Gallery with Effects**". It is a practical embodiment of Web Programming techniques using the languages HTML, CSS, JavaScript, and jQuery. The main concepts in focus are DOM accessing, DOM manipulation, and Animations.

Abstract

The Dynamic Image Gallery with Effects presents a lightweight yet feature-rich web application for real-time image collection management. Users can seamlessly **add images, remove** individual images, or clear the entire gallery with a single action.

Each operation is enhanced with smooth hybrid fade-slide **animations**, while a click-triggered zoom effect provides interactive visual feedback. The interface adapts **responsively** across devices through a CSS Grid-based layout.

Built with HTML5, CSS3, JavaScript, and jQuery, the project demonstrates practical **DOM** manipulation, custom animation sequencing, and graceful error handling for invalid image URLs.

The result is an intuitive, visually cohesive **gallery** that effectively integrates core web programming concept like event handling, dynamic rendering, and CSS transition within a clean single-page architecture.

Introduction

In current web programming, creating interactive user interfaces that respond **fluidly** to user actions is a necessary skill. Image galleries are a common real-world requirement found in portfolios, online stores, and content distribution systems. However, a static **gallery** that only displays images lacks engagement. The real value comes from **dynamic** behavior that is allowing users to add, remove, and interact with images while receiving smooth visual feedback.

Our project solves this problem by building a fully **interactive** web-based image gallery. Users can perform three main actions: **add new images** by entering a valid image URL, **remove** individual images using a remove button, or clear the entire gallery with a single click. Every action trigger custom **jQuery animation**, including a hybrid fade-slide effect during addition and removal, and a **zoom effect** when any image is clicked.

The core concepts driving this project are **Document Object Model (DOM)** manipulation and **jQuery animation** techniques. The application demonstrates how front-end technologies can be seamlessly integrated to produce a **responsive** and user-friendly gallery experience. The expected outcomes of this project include an **interactive** gallery with animated effects and real-time dynamic updates; all achieved without any page reload.

Objectives

The primary objectives of this project are as follows:

1. To design a visually appealing and **responsive** image gallery interface using HTML and CSS.
2. To implement dynamic **DOM manipulation** allowing users to add new images to the gallery via URL input and remove existing images without a page reload.
3. To apply jQuery **animation** effects, including custom fade and slide animations (fadeSlideToggle), for smooth insertion and removal of image cards.
4. To implement an interactive **zoom effect** on images using CSS class toggling triggered by jQuery click events.
5. To handle user **input validation** and error scenarios, such as invalid image URLs, with appropriate feedback.
6. To implement a gallery clear feature that removes all images with **animated transitions**.
7. To demonstrate a practical understanding of DOM, **event-driven** programming, and jQuery animation APIs in a real-world web application.

Key Features

- Responsive Design
- Error Handling
- Dynamic Layout
- Dynamic Image Handling
- Smooth Animations
- Real Time DOM Updates

Theory

Our project is built upon a foundation of core web programming technologies including HTML5, CSS3, JavaScript, and the jQuery library.

Technology stack :

- **HTML:** - Structural markup and content organization
- **CSS:** - Visual styling, layout, and responsive design
- **JavaScript:** - Core logic, event handling, and DOM manipulation
- **jQuery:** - Simplified DOM traversal, animation sequencing, and event binding

Hyper Text Markup Language (HTML) :

HTML is the standard markup language used to **structure content** on the web. It consists of a series of elements represented by tags, which define different parts of a document such as headings, paragraphs, links, images, forms, and containers. HTML provides the **skeletal framework** upon which all web pages are built. Each tag serves a specific semantic or structural purpose, and the nested arrangement of tags creates the **Document Object Model (DOM)**, a hierarchical tree representation of the page.

The Document Object Model (DOM) is a programming interface for web documents. It represents the page as a tree of objects, where each HTML element is a node that can be accessed and **manipulated using JavaScript**. The DOM allows scripts to dynamically change the structure, style, and content of a web page. In this project, DOM manipulation is central to adding new image cards, removing existing ones, and updating the gallery content in real time. Some HTML tags used: -

- **<header>** – Defines the top section of a page or section, typically containing titles, logos, or navigation.
- **<main>** – Specifies the main, central content of the document (unique and primary).
- **<div>** – A generic container used to group elements for styling or layout.
- **** – Embeds an image into the webpage.

Cascading Style Sheets (CSS) :

CSS is a stylesheet language used to define the **presentation** and layout of HTML documents. It controls how web pages **appear** by applying styles such as colors, fonts, spacing, alignment, and positioning to elements. By separating content (HTML) from design (CSS), it improves maintainability, consistency, and flexibility in web development.

CSS works on a **cascading** and **inheritance** principle, where multiple style rules can be applied to the same element, and conflicts are resolved based on priority, specificity, and order. Styles can be applied in different ways, such as inline, internal, or external stylesheets. CSS also supports **responsive design** through media queries, allowing web pages to adapt to different screen sizes and devices.

Important concepts in CSS include the **box model**, which consists of content, padding, border, and margin, and determines the layout and spacing of elements. Various positioning methods such as static, relative, absolute, fixed, and sticky help in placing elements accurately on the page. Modern layout techniques like **flexbox** and **grid** make it easier to design complex and responsive layouts.

CSS Grid is used to arrange image cards in a **responsive** multi-column layout that adjusts automatically to the screen width using auto-fit and minmax() values. CSS transitions provide **smooth** visual feedback for hover effects (card lift and shadow), and the zoom effect on images is implemented using CSS transform: scale() toggled by a jQuery-added class (.zoomed).

In CSS, selectors are patterns used to target HTML elements for styling. Types of CSS Selectors: -

- Element Selector – Targets all elements of a specific HTML tag.
- Class Selector – Targets elements that share the same class attribute.
- ID Selector – Targets a single unique element with a specific ID.
- Universal Selector – Targets all elements in the document.
- Group Selector – Targets multiple elements simultaneously.

JavaScript (JS) :

JavaScript is a high-level, interpreted programming language used to add interactivity and dynamic behavior to web pages. It works alongside HTML and CSS to create responsive and engaging user experiences by enabling features such as form validation, event handling, animations, and real-time content updates.

JavaScript is primarily **executed in the browser**, allowing it to manipulate webpage content through the Document Object Model (DOM). It can **dynamically modify** HTML elements, change styles, and respond to user actions like clicks, typing, and scrolling. Modern JavaScript also supports server-side development through environments like Node.js, making it a versatile full-stack language.

JavaScript follows an **event-driven** and object-based programming model, supporting concepts like variables, data types, functions, objects, and control structures such as loops and conditions. It also includes advanced features like closures, promises, and asynchronous programming, which allow efficient handling of tasks like API calls and background operations without blocking the main execution thread.

Another important aspect of JavaScript is its support for ES6 (ECMAScript 2015) and later features, including **arrow functions**, classes, modules, and improved syntax, which make the language more powerful and easier to use.

jQuery :

jQuery is a fast, **lightweight JavaScript** library designed to simplify HTML document manipulation, event handling, and animation. It provides an **easy-to-use syntax** that reduces the amount of code required compared to plain JavaScript. jQuery allows developers to **efficiently** select and manipulate elements using CSS-style selectors. It also simplifies tasks like handling events, making AJAX requests, and creating dynamic effects. Being cross-browser compatible, it ensures consistent behavior across different web browsers. Overall, jQuery makes web development faster and more convenient.

Algorithm

Each user interaction triggers a specific event handler, which manipulates the DOM and executes animations.

- **Document Ready Initialization: -**

Step 1: Start
Step 2: Wait for DOMContentLoaded event
Step 3: Define custom jQuery plugin fadeSlideToggle()
Step 4: Attach click event listener to "Add Image" button
Step 5: Attach keypress event listener to URL input field (for Enter key)
Step 6: Attach click event listener to "Clear Gallery" button
Step 7: Set up event delegation on gallery container for:
- Click events on .remove-btn elements
- Double-click events on .image-card elements
- Click events on img elements (for zoom)
Step 8: End

- **Add Image: -**

Step 1: Start
Step 2: Read URL from input field
Step 3: If URL is empty or null then:
 Step 3.1: Display alert "Please enter a valid URL"
 Step 3.2: Go to Step 10
Step 4: Create new div element with class "image-card"
Step 5: Create img element with src = URL
Step 6: Create button element with class "remove-btn" and text "✕"
Step 7: Append img and button to image-card div
Step 8: Set CSS display of image-card to "none" (hide)
Step 9: Append image-card to gallery container
Step 10: Call fadeSlideToggle(1200) on image-card to animate appearance
Step 11: Attach error event handler to img for invalid URL detection
Step 12: Attach click event to img for zoom functionality
Step 13: Attach double-click event to image-card for removal
Step 14: Clear input field value
Step 15: End

- **Remove Image: -**

Step 1: Start
Step 2: Triggered by click on .remove-btn OR double-click on .image-card
Step 3: Find closest ancestor element with class "image-card"
Step 4: Call fadeSlideToggle(1200) on the image-card element
Step 5: Wait for animation completion
Step 6: Remove image-card element from DOM
Step 7: End

- **Clear Gallery:**

Step 1: Start
Step 2: Count all elements with class "image-card"
Step 3: If count = 0 then:
 Step 3.1: Display alert "Gallery is already empty"
 Step 3.2: Go to Step 8
Step 4: Display confirmation dialog "Remove all images?"
Step 5: If user clicks Cancel then:
 Step 5.1: Go to Step 8
Step 6: Select all elements with class "image-card"
Step 7: For each image-card element:
 Step 7.1: Call fadeSlideToggle(1200)
 Step 7.2: On animation complete, remove element from DOM
Step 8: End

- **Zoom Animation: -**

Step 1: Start
Step 2: Triggered by click event on img element
Step 3: Toggle class "zoomed" on the clicked image
Step 4: If class "zoomed" is present then:
 Step 4.1: Apply CSS transform: scale(1.5)
 Step 4.2: Set cursor style to "zoom-out"
Step 5: Else:
 Step 5.1: Apply CSS transform: scale(1)
 Step 5.2: Set cursor style to "zoom-in"
Step 6: End

- **Invalid URL Error Handling: -**

Step 1: Start
Step 2: Triggered by image error event
Step 3: Display alert message "Invalid Image URL"
Step 4: Find parent element with class "image-card"
Step 5: Call fadeSlideToggle(1200) on parent card
Step 6: Wait for animation completion
Step 7: Remove parent card from DOM
Step 8: End

- **Event Delegation for Dynamic Elements: -**

Step 1: Start
Step 2: Attach event listener to static parent container (#gallery)
Step 3: User clicks on dynamically added .remove-btn
Step 4: Event bubbles up from child to parent
Step 5: jQuery checks if event target matches selector ".remove-btn"
Step 6: If match is found then:
 Step 6.1: Execute removeImage() function
Step 7: Else:
 Step 7.1: Ignore the event
Step 8: End

- **Animation Function (fadeSlideToggle): -**

Step 1: Start
Step 2: Extend jQuery prototype with method: fadeSlideToggle(speed, callback)
Step 3: Input parameters:
 - speed: Animation duration in milliseconds
 - callback: Optional function to execute after animation
Step 4: Call jQuery animate() method on selected element with:
 Step 4.1: opacity: 'toggle' (fades in or out)
 Step 4.2: height: 'toggle' (expands or collapses)
Step 5: Set animation duration = speed parameter
Step 6: If callback function is provided then:
 Step 6.1: Execute callback after animation completes
Step 7: End

Implementation

The project follows a clean, organized directory structure:

- **index.html** – The main HTML file containing the page structure including the header, URL input section, gallery controls, gallery grid container, and footer.
- **css/style.css** – The stylesheet defining the overall gradient background, card layout using CSS Grid, image zoom behavior using CSS transforms, hover effects, and animation-related CSS class (. zoomed).
- **js/script.js** – The jQuery-powered script implementing all dynamic behaviors: adding/removing images, animations, zoom toggle, and gallery clear functionality.

- **index.html:**

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Dynamic Image Gallery</title>
    <!-- CSS -->
    <link rel="stylesheet" href="css/style.css" />
    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
    <!-- JavaScript -->
    <script defer src="js/script.js"></script>
  </head>
  <body>
    <!-- ◇ Header Section -->
    <header>
      <h1>📷 Dynamic Image Gallery</h1>
      <p>Add, remove and interact with images using animations</p>

    </header>
    <!-- ◇ Upload Section -->
```

```

<section class="upload-section">
  <h2>Add New Image</h2>
  <input type="text" id="image-url" placeholder="Enter image URL here..." />
  <button id="add-btn">Add Image</button>
</section>
<!-- ◇ Gallery Controls -->
<section class="controls">
  <button id="clear-gallery">Clear Gallery</button>
</section>
<!-- ◇ Gallery Container -->
<main>
  <div id="gallery" class="gallery-container">
    <!-- Example Static Image (for testing UI) -->
    <div class="image-card">
      <img
        src=https://demo-res.cloudinary.com/image/upload/sample.jpg alt="Sample
Image"/>
      <button class="remove-btn">Remove</button>
    </div>
  </div>
</main>
<!-- ◇ Footer -->
<footer>
  <p>© 2026 Dynamic Gallery Project | Team Project</p>
</footer>
</body>
</html>

```

- css/style.css:

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Poppins", sans-serif;
}

```

```

body {
  background: linear-gradient(135deg, #003d4d, #00c996);
  color: #fff;
  min-height: 100vh;
}
/* Header */
header {
  text-align: center;
  padding: 30px 20px;
  background: rgba(255, 255, 255, 0.1);
  border-bottom: 3px solid rgba(0, 0, 0, 0.352);
}
header h1 {
  font-size: 2.5rem;
  margin-bottom: 10px;
}
header p {
  opacity: 0.8;
}
.upload-section {
  text-align: center;
  padding: 30px 20px;
}
.upload-section h2 {
  margin-bottom: 15px;
}
#image-url {
  padding: 12px;
  width: 60%;
  max-width: 400px;
  border: none;
  border-radius: 8px;
  outline: none;
  font-size: 1rem;
}
#image-url:focus {
  box-shadow: 0 0 20px rgba(255, 255, 255, 0.6);
}

```

```
button {
  padding: 12px 18px;
  margin-left: 10px;
  border: none;
  border-radius: 8px;
  background: #ff4b5c;
  color: white;
  letter-spacing: 0.5px;
  font-weight: bold;
  cursor: pointer;
  transition: 0.3s;
}
button:hover {
  background: #d90429;
  transform: scale(1.05);
}
/* Controls */
.controls {
  text-align: center;
  margin-bottom: 20px;
}
#clear-gallery {
  background: #ff4b5c;
}
#clear-gallery:hover {
  background: #d90429;
}
/* Gallery */
.gallery-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
  padding: 20px;
}
/* Image Card */
.image-card {
  display: none;
  border-radius: 15px;
```

```

overflow: hidden;
box-shadow: 0 8px 25px rgba(0, 0, 0, 0.253);
transition: transform 0.4s ease, box-shadow 0.4s ease;
position: relative;
}
.image-card:hover {
  transform: translateY(-8px) scale(1.03);
  box-shadow: 0 12px 35px rgb(0, 0, 0);
}
/* Image */
.image-card img {
  width: 100%;
  height: 230px;
  object-fit: cover;
  display: block;
  transition: transform 0.4s ease, box-shadow 0.4s ease;
  cursor: zoom-in;
}
.image-card img.zoomed {
  transform: scale(1.5);
  cursor: zoom-out;
  box-shadow: 0 0 20px rgba(0, 0, 0, 0.6);
}
.remove-btn {
  position: absolute;
  bottom: 10px;
  right: 10px;
  background: rgba(255, 75, 92, 0.9);
  padding: 6px 12px;
  font-size: 0.9rem;
}
.remove-btn:hover {
  background: #d90429;
}
footer {
  text-align: center;
  padding: 20px;
  margin-top: 20px;
}

```

```
background: rgba(0, 0, 0, 0.288);
font-size: 0.9rem;
opacity: 0.8;
}
```

- js/script.js:

```
$(document).ready(function () {
  // Custom animation: fade + slide together
  $.fn.fadeSlideToggle = function(speed, callback) {
    return this.animate({
      opacity: 'toggle',
      height: 'toggle'
    }, speed, callback);
  };
  function addImage() {
    var url = $('#image-url').val().trim();
    if (!url) {
      alert('⚠ Please enter an image URL.');
```

return;

```
    }
    var newCard = $(
      '<div class="image-card">' +
        '' +
        '<button class="remove-btn">Remove</button>' +
        '</div>'
    );
    // Apply hybrid animation
    newCard.hide();
    $('#gallery').append(newCard);
    newCard.fadeSlideToggle(1200);
    // Handle image load error
    newCard.find('img').on('error', function () {
      newCard.fadeSlideToggle(1200, function () {
        $(this).remove();
      });
    });
  }
});
```

```

        alert('✖ Invalid image URL.');
```

});

// Zoom effect

```

newCard.find('img').on('click', function () {
    $(this).toggleClass('zoomed');
});
// Double-click to remove
newCard.on('dblclick', function () {
    $(this).fadeSlideToggle(1200, function () {
        $(this).remove();
    });
});
$('#image-url').val("");
}
// Remove with hybrid animation
function removeImage(button) {
    $(button).closest('.image-card').fadeSlideToggle(1200, function () {
        $(this).remove();
    });
}
// Render gallery with hybrid effect
function renderGallery(imageArray) {
    $('#gallery').empty();
    $.each(imageArray, function (index, url) {
        var $card = $(
            '<div class="image-card">' +
            '' +
            '<button class="remove-btn">Remove</button>' +
            '</div>'
        );
        $('#gallery').append($card);
        $card.hide().fadeSlideToggle(1200);
        $card.find('img').on('click', function () {
            $(this).toggleClass('zoomed');
        });
    });
}
// Event Listeners
```

```

$('#add-btn').on('click', addImage);
$('#image-url').on('keypress', function (e) {
    if (e.which === 13) addImage();
});
$('#gallery').on('click', '.remove-btn', removeImage);
$('#clear-gallery').on('click', function () {
    if ($('#image-card').length === 0) {
        alert('❗ Gallery is already empty.');
```

return;

```

    }
    if (confirm('🗑️ Clear all images?')) {
        $('#image-card').fadeToggle(1200, function () {
            $(this).remove();
        });
    }
});
// Initialize zoom
$('#gallery img').on('click', function () {
    $(this).toggleClass('zoomed');
});
});

```

Result

The Dynamic Image Gallery application was successfully implemented and tested. The following results were observed:

- **Image Addition:** Users can enter any valid image URL in the input field and click "Add Image" or press Enter. The image card appears in the gallery with a smooth fade-and-slide animation (duration: 1200ms).
- **Image Removal:** Clicking the Remove button on a card or double-clicking the card causes it to animate out (fade-and-slide collapse) and be removed from the DOM. No page reload is required.
- **Invalid URL Handling:** If an invalid image URL is entered, the card briefly appears and then animates out, and an alert informs the user of the error.
- **Clear Gallery:** The "Clear Gallery" button removes all images simultaneously with animation after user confirmation. If the gallery is already empty, an informational alert is displayed.
- **Responsive Layout:** The gallery grid adapts automatically to different screen sizes, arranging cards in 1, 2, 3, or more columns depending on the available viewport width.

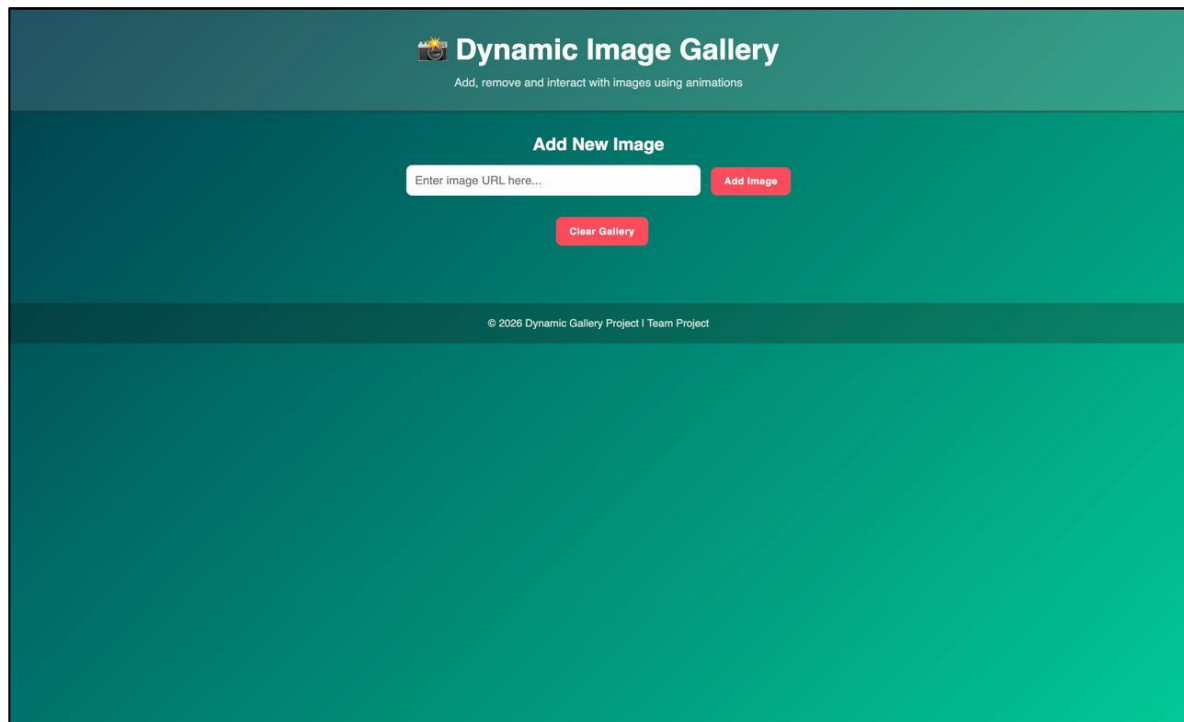


Fig 1: - Empty Gallery

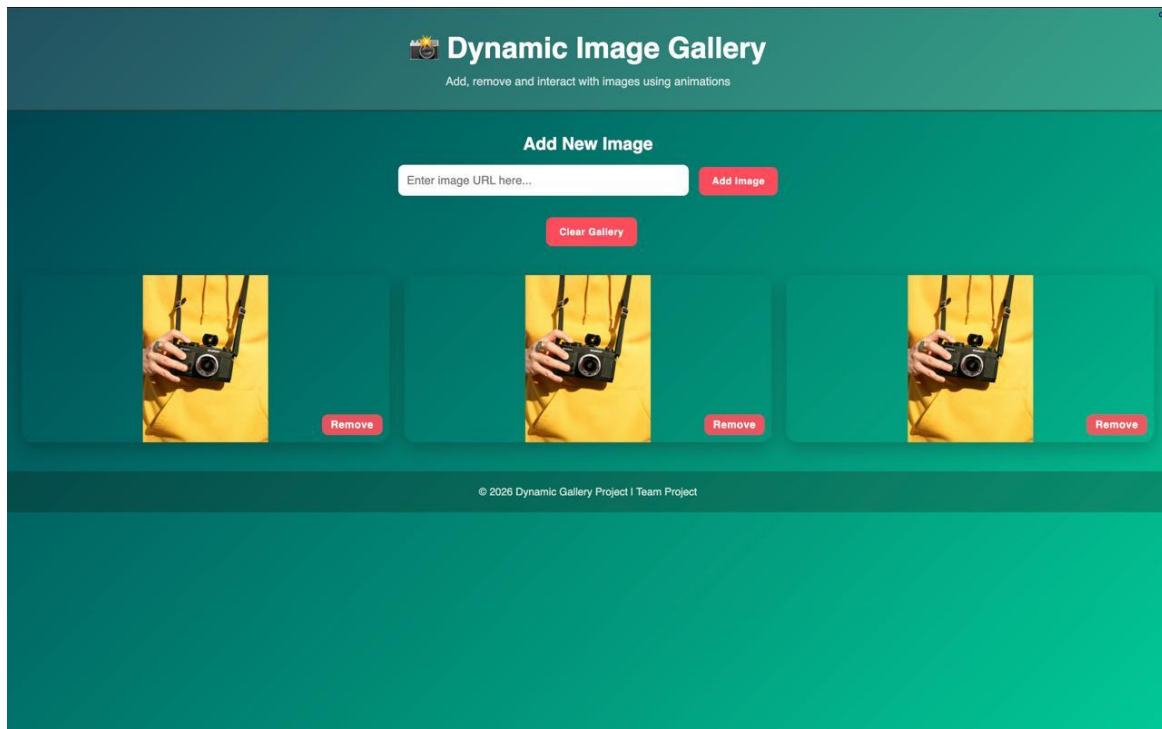


Fig 2: - Gallery after adding three images

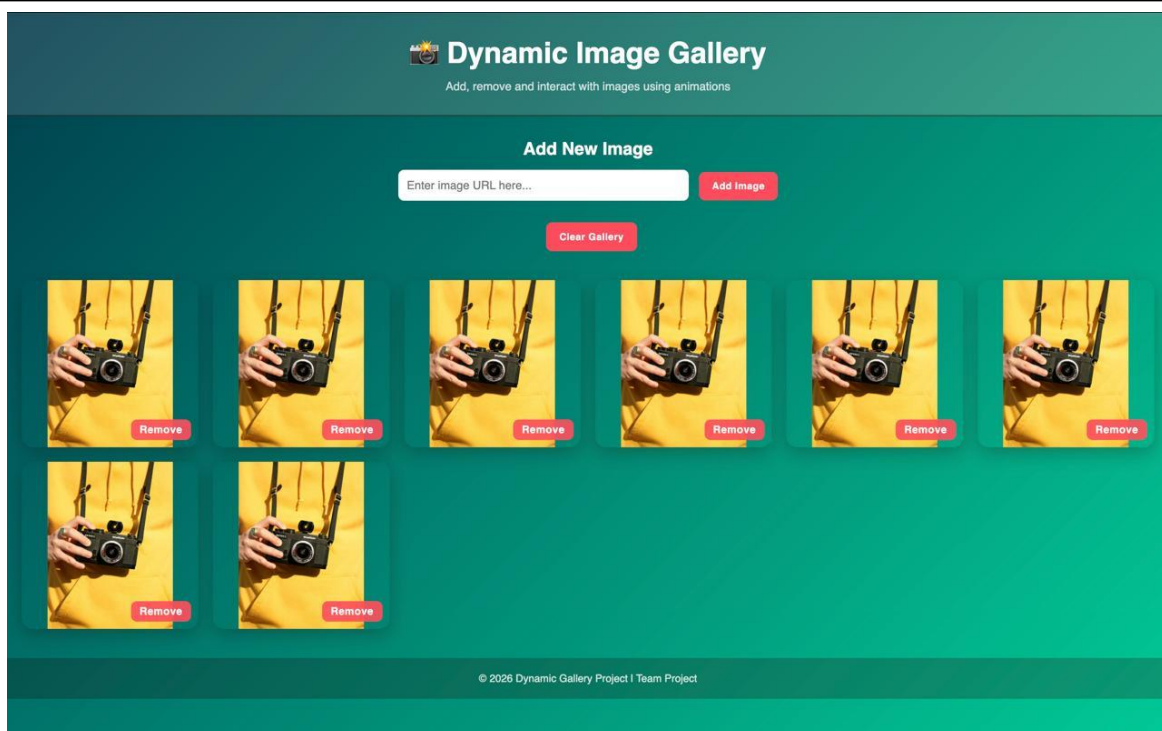


Fig 3: - Gallery after Multiple images

Conclusion

Our project successfully demonstrates the practical application of core web programming concepts including DOM manipulation, event-driven programming, and jQuery-based animations. The project achieves its primary objective of building an interactive web application where users can dynamically add, remove, and interact with images in real time.

The final application delivers all features including image addition via URL, individual removal, gallery clearing, click-to-zoom, double-click removal, and responsive layout. Each operation is accompanied by smooth visual feedback, enhancing the overall user experience. The project also handles edge cases such as empty URL submission and invalid image links without crashing or disrupting existing gallery content.

Individual Contribution

The successful completion of Our Effects project was made possible through the collaborative efforts of all team members.

Name	Role	Contribution
Jaskaran Singh	jQuery Effects	Created animation function, and implemented event delegation
Prashant Kumar Mishra	Documentation	Prepared the project report and documented the compiled references
Bhanu Katoch	HTML Structure	Designed the overall HTML document structure and project planning
Ajay Kumar	CSS Design	Developed all visual styling including responsive grid design
Nikita Mewada	JavaScript Logic	Implemented DOM manipulation and event handling
Saptarishi Das	Integration & Testing	Integrated all modules, performed testing, and resolved integration issues

REFERENCES

1. ThomasA. Powell- HTML & CSS: The Complete Reference, TMH, New Delhi
2. Ryan Benedetti, Ronan Cranley- Head First jQuery-A Brain-Friendly Guide, O'Reilly Media
3. Matthew David- HTML5, O'Reilly Media
4. Keith, J., & Sambells, J. (2010). DOM scripting: Web design with JavaScript and the document object model (2nd ed.). Apress.
5. Johanan, J. (2016). Web developer's reference guide. Packt Publishing.
6. <https://www.w3schools.com/html/>
7. <https://developer.mozilla.org/en-US/docs/Web/HTML>
8. <https://www.geeksforgeeks.org/css/css-tutorial/>
9. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
10. <https://jquery.com/>